RAG-Based Customer Support Assistant using LangGraph & HITL

TECHNICAL DOCUMENTATION

Objective

The objective of this Technical Documentation is to provide a detailed engineering-level explanation of the complete RAG-Based Customer Support Assistant system. This document explains architecture design decisions, workflow execution, retrieval logic, LangGraph orchestration, HITL integration, implementation strategies, testing approaches, and future scalability plans.

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval with generative language models. Instead of depending only on pre-trained knowledge, the system retrieves relevant external information from documents and uses that information to generate accurate and context-aware responses.

RAG improves reliability, reduces hallucinations, and enables AI systems to answer questions based on domain-specific knowledge.

Why RAG is Needed

Traditional chatbots often generate inaccurate or generic responses because they lack access to real-time or organization-specific data.

RAG systems solve this problem by:

- Retrieving relevant knowledge dynamically
- Improving answer accuracy
- Supporting enterprise document systems
- Reducing hallucinated responses

Use Case Overview

This project focuses on a customer support assistant that answers user queries using information extracted from PDF documents.

Example queries include:

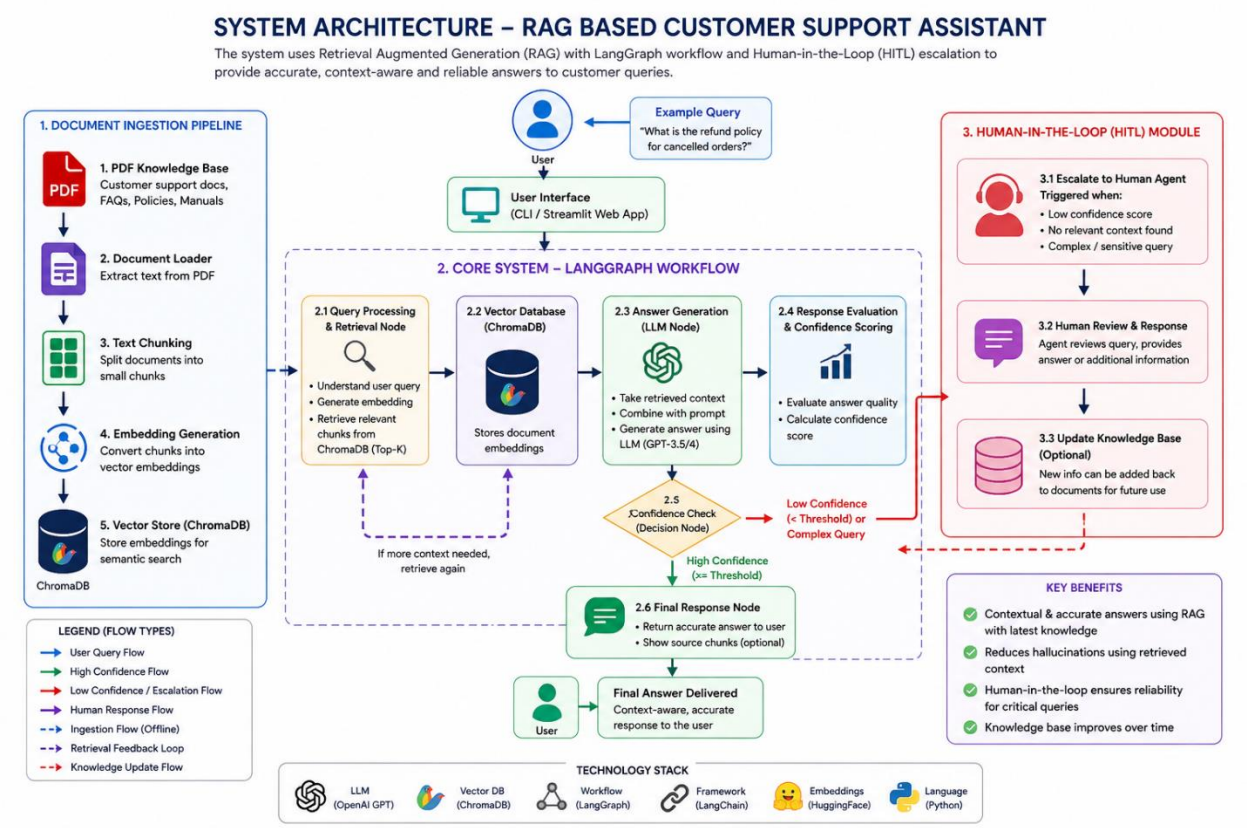
- What is the refund policy?
- How can I reset my password?
- What are the delivery timelines?

If the system cannot confidently answer, it escalates the query to a human agent.

2. System Architecture Explanation

Detailed System Architecture

The RAG-Based Customer Support Assistant follows a layered architecture where each layer performs a dedicated responsibility in the retrieval and response generation pipeline.



Frontend Layer

The frontend layer acts as the interaction point between users and the AI system. It accepts user queries and displays generated responses.

Document Processing Layer

This layer handles PDF ingestion, text extraction, and chunk creation. It converts raw PDF data into structured chunks suitable for embedding generation.

Embedding Layer

The embedding layer transforms chunks into vector representations using Sentence Transformers. These embeddings capture semantic meaning rather than simple keywords.

Vector Database Layer

ChromaDB stores embeddings efficiently and performs similarity search during retrieval.

Retrieval Layer

The retriever identifies the most relevant chunks based on semantic similarity between user queries and stored embeddings.

LLM Layer

The Large Language Model receives retrieved context and generates human-like responses grounded in document knowledge.

LangGraph Workflow Layer

LangGraph controls node execution, state transitions, and routing logic across the workflow.

HITL Layer

The Human-in-the-Loop module manages escalations for uncertain or complex queries requiring human intervention.

3. Design Decisions

Chunk Size Choice

Chunk size directly impacts retrieval quality.

Small Chunks

Advantages:

- Better retrieval precision

- Faster embedding generation

Disadvantages:

- Loss of context

Large Chunks

Advantages:

- Better contextual understanding

Disadvantages:

- Increased token usage
- Lower retrieval precision

Chosen Configuration:

- Chunk Size: 500
- Overlap: 100

This provides balanced retrieval quality and contextual continuity.

Embedding Strategy

The project uses Sentence Transformers because they provide:

- Fast embedding generation
- Strong semantic understanding
- Lightweight deployment

Model Used:

- all-MiniLM-L6-v2

Retrieval Approach

Semantic similarity search is used instead of keyword matching because:

- It understands intent
- Handles paraphrased queries
- Improves contextual relevance

Prompt Design Logic

Prompt templates are designed to:

- Restrict hallucinations
- Force context-based answers
- Encourage concise responses

Example Prompt:

Answer the question using only the provided context.

If information is unavailable, say that escalation is required.

4. Workflow Explanation

LangGraph Usage

LangGraph enables graph-based execution using nodes and edges.

Advantages:

- Modular architecture
- Flexible routing
- Stateful workflows

Node Responsibilities

Input Node: Receives user query.

Retrieval Node: Fetches relevant chunks.

Generation Node: Generates contextual response.

HITL Node: Handles escalation.

Output Node: Returns final response.

State Transitions

State carries information between nodes.

Example State:

state = {

"query": query,

"retrieved_docs": docs,

"response": response,

"confidence": confidence }

5. Conditional Logic

Intent Detection

The system analyzes:

- Query complexity
- Missing context
- Sensitive intent

Routing Decisions

Normal Query

Route:

Query -> Retrieval -> LLM -> Output

Complex Query

Route:

Query -> Retrieval -> Confidence Check -> HITL

6. HITL Implementation

Role of Human Intervention

Human intervention ensures:

- Reliability

- Trustworthiness
- Resolution of ambiguous queries

Benefits

- Improved customer satisfaction
- Reduced AI hallucinations
- Better handling of edge cases

Limitations

- Increased operational cost
- Human dependency
- Slower response time

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed

Larger retrieval sizes improve accuracy but increase latency.

Chunk Size vs Context Quality

Small chunks improve precision but may lose contextual continuity.

Cost vs Performance

Advanced LLMs improve quality but increase API costs.

8. Testing Strategy

Functional Testing

- PDF ingestion testing

- Query testing
- Retrieval accuracy testing

Sample Queries

Query	Expected Result
What is the refund policy?	Retrieve refund information
How to reset password?	Retrieve account support steps
Explain billing issue	HITL escalation

Performance Testing

- Query latency
- Retrieval speed
- LLM response time

9. Future Enhancements

Multi-document Support

Enable multiple PDF ingestion.

Feedback Loop

Allow users to rate responses.

Memory Integration

Maintain conversational memory.

Cloud Deployment

Deploy using AWS/GCP/Azure.